

# Bhavy Yadav

Go Backend Engineer · Distributed Systems · Security Infrastructure · Linux · Bhavyadav25.github.io  
+91 7303345356 · yadavbhavy25@gmail.com · github.com/Bhavyadav25 · linkedin.com/in/yadavbhavy · Delhi, India

---

## Summary

Go backend engineer with 2.5 years in production security infrastructure — DDoS detection, enterprise WAF, and an ICAP server processing 100k+ daily HTTP transactions at WiJungle. Writes open-source Go tooling: a zero-allocation structured logger that outperforms zap 3.2×, and a drop-in observability library for Go HTTP services. IIT Delhi graduate.

## Education

### Indian Institute of Technology (IIT) Delhi

2019 – 2023

B.Tech, Fiber Science & Nanotechnology · State-level badminton player · Cross-functional team lead (35 members)%

## Skills

**Go:** goroutines, channels, sync.Pool, atomic ops, net/http, zero-alloc design, pprof, benchmarking

**Backend:** Kafka, MQTT, REST, GraphQL, gRPC, WebSockets, event-driven, microservices

**Databases:** PostgreSQL, ScyllaDB, Manticore Search

**Infrastructure:** Linux, NGINX, Squid, Unix sockets, Docker, AWS, CI/CD

**Security:** ModSecurity (WAF), DDoS detection, ICAP (RFC 3507), TLS inspection

**Languages:** Go (primary), Java, Python, C++

## Experience

### Clickpost Software Development Engineer — Backend

Sep 2025 – Feb 2026

*Logistics tech, Remote · Java/Python microservices for hyper-local delivery*

- Replaced a synchronous API mesh (which was causing cascading failures during bulk store ingestion) with Python Kafka consumer pipelines — dead-letter queuing for poison-pill handling, decoupled service boundaries. 85% fewer manual API calls.
- Developed a Java-based store management platform: Haversine geolocation for HLD delivery radius and carrier route eligibility, bulk CRUD across 1,000+ stores.

### WiJungle Software Engineer — Backend Systems

Jul 2023 – Aug 2025

*Cybersecurity startup, Gurugram · Core backend team of 4*

- Architected a Go DDoS detection engine from scratch — goroutine worker pools, sliding-window traffic counters, adaptive threshold rules, automated mitigation triggers. Detection latency down 35%, customer downtime down 60%.
- Owned the WAF end-to-end: Go rule orchestration service with hot-reload via UNIX API, per-client ModSecurity rulesets, false-positive tuning under production load. 50+ client environments; breach incidents down 25%.
- Implemented an RFC 3507-compliant Go ICAP server, integrated with Squid: streaming I/O, connection pooling, 100k+ daily transactions. Root cause was proxy timeouts under burst load; the server eliminated them without restarting upstream services.
- Designed a high-throughput log normalization pipeline in Go: 6 input formats → structured JSON → Unix socket → Manticore Search, with Prometheus counters for pipeline lag alerting. Processing time down 80%.
- Engineered a C++ GraphQL parser and integrated it with ModSecurity as an adaptive module, achieving a significant reduction in parsing latency by 40%, while ensuring robust data integrity throughout the process.
- Led 3-engineer team configuring 20+ production routers. Built a Go + MQTT config state machine for push-based policy distribution; eliminated 70% of manual ops work, uptime up 15%.

## Open Source

### loghq Go · github.com/Bhavyadav25/loghq

Zero-allocation structured logger. Hand-rolled JSON, logfmt, and console encoders; sync.Pool recycling; inline [16]Field array keeps the hot path off the heap; atomic level check at ~3ns. Benchmarked: 320ns/op (0 allocs) vs zap's 1012ns/op — 3.2× faster with no allocations vs zap's 320B/op. Includes file rotation with gzip, context propagation, multi-handler routing.

### xrayhq Go · github.com/Bhavyadav25/xrayhq

Drop-in observability for Go HTTP services. Instruments net/http, Chi, Echo, Gin, Fiber + database/sql, GORM, Redis (go-redis), MongoDB. Real-time SSE dashboard, ring buffer storage, automatic alerting for N+1 queries, slow routes (P95), error spikes, and panics. Zero external infrastructure.